Contract-Governed Adversarial Evaluator Hardening: Stage-Gated Recursive Improvement with Typed Promotion Contracts

Daniel Alami*
Independent Researcher
MBA Candidate, Harvard Business School

Abstract

When an adversarial evaluator is itself the object of recursive improvement, unconstrained optimization can soften the enforcement surface the evaluator is supposed to maintain. This paper describes a stage-gated architecture in which a deterministic meta-runner—no learned parameters, no language-model judgment—governs evaluator-hardening by executing precommitted Python promotion contracts that return PASS, FAIL, or BLOCKED verdicts. Over a four-month development period, six evaluator-hardening stages were promoted through this mechanism. The contracts blocked one premature promotion before a fix was applied, scoped each stage to a named evaluation surface so that no stage could claim credit for improvements made elsewhere, and forced integration debts into separately governed programs rather than letting them inflate passing stage claims. The work builds on prior documentation of specification gaming in LLM-generated code [Alami, 2025a] and evaluator hardening through mined failure constraints [Alami, 2025b]. All claims are scoped to one human-operated research system; generalization requires independent replication.

1 Introduction

As language model systems become capable of multi-step recursive operation, a practical question arises: how should improvements to the evaluation layer itself be governed? If the same optimization pressure that produces specification gaming in agent outputs [Alami, 2025a] also applies to the evaluator—and prior work suggests it does [Alami, 2025b]—then unconstrained recursive improvement of the evaluator is structurally dangerous. The evaluator is both the object of improvement and the judge of whether improvement succeeded.

This paper presents a stage-gated architecture that governs recursive evaluator hardening through precommitted Python promotion contracts. Each stage in the improvement queue has a typed contract that the meta-runner evaluates before allowing advancement. The meta-runner is intentionally simple: it executes contracts, not interpretations. FAIL and BLOCKED are hard stops that require human diagnosis before continuation.

The contribution is not evaluator hardening itself—that is the subject of prior work [Alami, 2025b]. The contribution is governance of evaluator hardening: a deterministic layer that controls what improvements are allowed to promote, on which evaluation surface, and with what evidence.

*SSRN abstract ID: 6542998

1.1 Threat Model for Recursive Evaluator Improvement

Four failure modes arise when evaluator improvements are unconstrained:

1. **Score-chasing.** The improving agent optimizes for the evaluator's numeric output rather than its discriminative quality. The evaluator becomes lenient; games that should fail begin to pass.
2. **Scope creep.** An improvement to one evaluator capability (e.g., primitive routing) claims credit for improvements in another capability (e.g., gate stabilization). Attribution is laundered and individual stage contributions cannot be assessed.
3. **Silent debt absorption.** Integration issues between stages are absorbed into passing stage claims rather than being surfaced. The result is fragile improvements that break under composition.
4. **Evaluator softening.** The improving agent modifies the evaluator in a way that makes its own future improvements easier to promote. The enforcement surface ratchets toward leniency rather than hardness.

The meta-runner addresses all four: contracts check typed properties rather than scores (1); promotion-path scoping restricts which evaluation surface counts (2); debt externalization forces cross-stage debts into separate programs (3); and the meta-runner has no parameters to optimize—its contracts are precommitted Python (4).

2 Related Work

2.1 Constitutional AI

Constitutional AI [Bai et al., 2022] governs model outputs through a recursive critique-revise loop with an LLM judge. The improvement signal is linguistic at each step. There is no precommitted promotion contract; the loop continues until a human or scheduler stops it. The meta-runner differs in two respects: the object of improvement is the adversarial evaluator, not the policy; and promotion is governed by deterministic Python contracts, not by LLM-derived judgment.

2.2 Reflexion

Reflexion [Shinn et al., 2023] accumulates verbal self-feedback across episodes to improve agent behavior—the agent judges its own prior performance in natural language, and the improving system and the evaluation of improvement are not structurally separated. The meta-runner sits outside the improvement loop entirely: its promotion layer is non-linguistic, its verdicts (PASS / FAIL / BLOCKED) are typed hard stops, and its contract code is replayable by any reviewer. Whether this separation is practically safer than Reflexion-style loops has not been tested in a controlled comparison. The claim here is architectural: the governance properties are verifiable systems results encoded in deterministic code, not behavioral observations that depend on a specific benchmark outcome.

2.3 Process Supervision

Process reward models [Lightman et al., 2023] score intermediate reasoning steps rather than only final outputs, improving evaluation granularity. The distinction from the meta-runner is that process reward models use a learned scorer whose parameters are optimized and whose outputs can drift under distributional shift. The meta-runner's contracts are deterministic Python with no learned

parameters. A process reward model improves signal quality; the meta-runner enforces a stable promotion floor.

2.4 CI/CD and Standard Software Testing

The most likely objection to the meta-runner is that it resembles standard CI/CD: run typed tests, gate deployment on pass/fail results. The resemblance is real and the paper should not obscure it. The structural distinction is that in CI/CD, the test suite is not the code being deployed—generation and evaluation are separated at the organizational level. In recursive evaluator improvement, the evaluator is both the object of improvement and the system that judges improvement quality. This reflexive property means that unconstrained optimization of the evaluator can degrade the enforcement surface itself—a failure mode that standard CI/CD does not face because the deployment artifact and the test suite are independent codebases.

3 Architecture

3.1 System Overview

The system has three layers:

1. **Evaluation kernel.** The adversarial evaluator that judges LLM-generated theses through deterministic score gates, adversarial precedent memory, and a structured adversarial review committee. This is the object of recursive improvement. Its architecture and hardening methodology are described in Alami [2025a,b].
2. **Meta-Runner.** A deterministic orchestrator that manages a queue of evaluator-hardening stages. Each stage has a named promotion contract in a Python registry. The meta-runner calls the contract, receives a typed verdict (PASS, FAIL, or BLOCKED), and advances or halts accordingly. It has no learned parameters and makes no LLM calls.
3. **Stage Queue.** A fixed sequence of six evaluator-hardening stages, each with a priority level (P0 or P1) and a named contract. The queue is defined before execution begins; the meta-runner cannot reorder, add, or remove stages.

3.2 Contract Interface

Each promotion contract is a Python function with the signature:

```
(project: str, benchmark_results: Any) -> ContractResult
```

A `ContractResult` contains:

- `verdict`: one of `"pass"`, `"fail"`, `"blocked"`
- `reasons`: a list of typed strings explaining each check
- `details`: optional structured metadata

The meta-runner's `advance()` method raises a `RuntimeError` if the current stage's verdict is not `"pass"`. There is no override, no soft promotion, no LLM-mediated exception.

3.3 Verdict Semantics

- **PASS.** All contract checks satisfied. The meta-runner may advance to the next stage.

- **FAIL.** At least one contract check failed. The stage cannot promote. The failure reasons are typed and archived. Human diagnosis is required before retrying.
- **BLOCKED.** All local checks pass, but required benchmark evidence is absent or incomplete. The stage is structurally sound but cannot promote until external evidence is supplied. This is the evidence-blocked state—governance without performance data.

3.4 Promotion-Path Scoping

Each stage's benchmark evidence includes a `promotion_path` field that specifies which evaluation surface the stage is judged against. The contract verifies that the promotion path matches the stage's scope. A stage that improves primitive routing cannot promote by pointing at deterministic-gate improvements made in earlier stages. The contract enforces this by checking the `promotion_path` field in the benchmark evidence and rejecting mismatches. This prevents attribution leakage between stages. The specific promotion paths per stage are shown in the governance table (Section 4.2).

3.5 Debt Externalization

When a stage's contract identifies integration debt that crosses stage boundaries, the debt becomes a separately governed program rather than being absorbed into the stage's passing claim. Concretely: if Stage N's contract detects that an upstream seam is unreliable, the debt is documented in the benchmark evidence and routed to a new program—not silently inherited by downstream stages. The contract encodes what a stage is *not allowed to claim*, not just what it must demonstrate.

4 Experiments

4.1 Overview

The meta-runner governed six evaluator-hardening stages for the `epistemic_engine_v4` project. All six stages promoted with benchmark evidence, fixture regressions, and typed verdicts. The full governance record is preserved in benchmark evidence JSON files, one per stage.

4.2 Governance Table

Stage	Promotion Path	1st	Rerun	Contract Checks	De Ex
1 — Semantic gates	`B_det_gates`	PASS	—	frozen match, symbols, compile, bench (7), OOD	No
2 — Hinge extraction	`B_det_gates`	**FAIL**	PASS	frozen match, hinge ifaces, compile, bench (4)	No
3 — Primitive routing	`C_gates+prims`	PASS	—	frozen match, routing ifaces, compile, bench (6)	No
4 — Shadow board	`handoff_fixture`	PASS	—	frozen match, fixture (8), handoff symbols, bench (7)	Ye
5 — Info yield	`loop_ctrl_fixture`	PASS	—	frozen match, fixture (9), loop symbols, bench (6)	No
6 — Cross-domain	`transfer_fixture`	PASS	—	frozen match, fixture (6), transfer symbols, bench (6)	Ye

Table 1: Governance table for six evaluator-hardening stages. Stage 2's first-run FAIL is the primary enforcement exhibit. Promotion paths vary per stage scope. Debt externalization is noted for Stages 4 and 6.

4.3 Enforcement Exhibit: Stage 2 FAIL

Stage 2 (`load_bearing_hinge_extraction`) is the primary enforcement exhibit. On run `20260405_191220`, the contract returned **FAIL**. The failure reason: "B blocks on deterministic_score_contract." The issue was a boundedness/input-domain problem—the hinge extraction logic produced incorrect results on a boundary specimen, not a hinge-classification confusion.

The meta-runner's `advance()` method refused to proceed. The failure reasons were archived. After the issue was diagnosed and fixed, a second run (`20260405_192002`) returned PASS on all mandatory specimens, and the stage promoted.

This is a documented case where the contract blocked a promotion that would have proceeded under unconstrained iteration. The failure was typed, timestamped, and the fix was independently verifiable through the benchmark evidence diff between the two runs.

4.3.1 BLOCKED States During Development

Before benchmark evidence was collected, each stage passed its local architecture checks (frozen candidate match, typed symbol presence, compile, runtime) but could not promote because the benchmark evidence file did not yet exist. The contract returned BLOCKED with reasons such as: "Local stage-1 architecture checks pass, but no `stage1_benchmark_evidence.json` exists yet. Stage 1 cannot promote until benchmark evidence shows no CLEAR/FATAL regression." This is the evidence-blocked state described in Section 3.3: governance without performance data. The meta-runner's `advance()` method refused to proceed in exactly the same way as for a FAIL verdict—no override, no exception.

4.4 Attribution Exhibit: Promotion-Path Scoping

Stage 1 promoted on `B_deterministic_gates`—the narrowest evaluation surface that isolates semantic-gate behavior. The benchmark evidence records the decision: "Stage 1 is scoped to semantic-gate stabilization. Promotion condition is `B_deterministic_gates` only."

When Stage 3 (primitive routing) was evaluated, the contract required promotion on `C_gates_plus_primitives`—a different evaluation surface—because routing only affects primitive-enabled evaluation. The contract checks this by verifying `promotion_path` in the benchmark evidence. If Stage 3 had submitted evidence on the `B_deterministic_gates` surface, the contract would have returned FAIL.

This scoping prevented Stage 3 from claiming credit for gate improvements made in Stages 1–2. Each stage's contribution is independently attributable because the promotion surface is fixed per stage.

4.5 Debt Externalization Exhibit

Stage 4 (shadow board taxonomy) needed a typed `Stage2Handoff` to wire the hinge object into the committee assignment path. The seam between hinge extraction and committee routing carried integration debt: extraction fidelity was not guaranteed for all input shapes.

The contract architecture forced this debt into two separately governed programs rather than letting Stage 4 absorb it:

- A **bridge audit program** to verify the typed handoff between hinge extraction and committee routing.

- A **derivation seam hardening program** to harden the function that constructs hinge objects from raw text, addressing fabrication paths in the extraction logic.

The benchmark evidence makes the boundaries explicit. Stage 4's record states: "Upstream extraction fidelity debt remains explicit; stage 4 must not pretend it is solved." Stage 6's record states: "The Stage 2→4 bridge debt must not be silently inherited by Stage 6; unresolved dependency should route to manual review." The debt was not eliminated—it was made visible and prevented from inflating stage claims.

4.6 Summary

Across six stages, the contracts produced governance artifacts at three levels: enforcement (Stage 2 FAIL blocked a regression), attribution (promotion-path scoping fixed what counts as evidence per stage), and scope discipline (debt externalization encoded what each stage is *not allowed to claim*). These are not post-hoc interpretations—they are properties of the precommitted contract code, replayable against the archived benchmark evidence.

5 Interpretation

5.1 Why Dumb Orchestrators Are Structurally Safer

The meta-runner has no learned parameters and cannot be optimized against. Score-chasing gains nothing because the meta-runner does not use scores—it checks typed contract conditions. Scope creep is blocked by promotion-path scoping. Silent debt absorption is blocked by the externalization protocol. Each of the four threat-model failure modes (Section 1.1) maps to a structural countermeasure rather than a behavioral hope.

The claim is not that parameterless orchestrators are always better than intelligent ones. For the specific problem of governing recursive evaluator improvement—where Goodhart reflexivity is structurally present—a parameterless orchestrator removes the optimization surface that a learned orchestrator would expose. The governance properties in Section 4 are architectural, encoded in replayable contract code and inspectable by any reviewer who reads the source.

5.2 Relationship to Prior Work in This System

This paper extends the evaluator hardening methodology documented in Alami [2025b] by adding a governance layer above it. The relationship is:

- Alami [2025a] documented the threat: specification gaming in LLM-generated code.
- Alami [2025b] developed the defense: adversarial precedent memory and failure→constraint→retest.
- This paper governs the defense: typed contracts that control what improvements promote and on which surface.

The contribution is the governance architecture for the improvement process itself—evaluator gaming is already established in the prior work.

6 Limitations

This paper rests on a single research system, a single target project, and a single codebase. That is the central limitation. Additional limitations:

(a) Six stages promoted on one project (`epistemic_engine_v4`). The architecture has not been tested on a different evaluator, a different domain, or by an independent team. Generalization requires independent replication.

(b) The primary enforcement exhibit is one FAIL verdict. The remaining five stages passed on their first decisive run. A larger corpus of enforcement events would strengthen the governance claim.

(c) The comparison with unconstrained recursive loops is architectural, not empirical. This paper does not include a controlled experiment in which the same six stages are run without contracts and the outcomes compared. The claim is that the contracts provide structural governance properties (typed enforcement, promotion-path scoping, debt externalization), not that they produce empirically superior evaluator quality in a head-to-head comparison.

(d) The meta-runner's contracts are written by the system designer. If a contract is flawed, the meta-runner executes it faithfully. The system bottlenecks the alignment problem at the contract-writing step rather than solving it. This is an explicit design choice—deterministic execution of human-authored contracts is preferred over probabilistic judgment—but it means the governance floor is only as good as the contracts that define it.

(e) The system does not yet demonstrate multi-operator governance. Concurrent review, delegated contract signing, and independent promotion authority require architectural extension.

7 Conclusion

Recursive improvement of adversarial evaluators requires a governance layer that the improving agent does not control. Without one, the evaluator that judges improvement quality can be softened by the same optimization pressure it is supposed to resist.

The meta-runner is a deliberately simple response: a deterministic orchestrator with typed promotion contracts, three verdict types, and no learned parameters. Across six evaluator-hardening stages, the contracts blocked a premature promotion, scoped each stage to its named evaluation surface, and forced integration debts into separately governed programs.

This is not a universal governance framework. It is a concrete, replicable architecture for a specific problem—governing recursive evaluator improvement without introducing a new optimization surface—transferable to any domain where the evaluation layer is itself the subject of recursive change.

References

Daniel Alami. Cognitive camouflage: Specification gaming in LLM-generated code evades holistic evaluation but not adversarial execution, 2025a. Working paper.

Daniel Alami. Adversarial precedent memory: Hardening LLM evaluators through mined failure constraints, 2025b. Working paper.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback, 2022.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.